

# Chapter 1

## $n$ -step Bootstrapping

### Contents

|     |                                                                                  |   |
|-----|----------------------------------------------------------------------------------|---|
| 1   | Introduction . . . . .                                                           | 2 |
| 2   | $n$ -step TD Prediction . . . . .                                                | 2 |
| 2.1 | $n$ -step Return . . . . .                                                       | 2 |
| 2.2 | Algorithm . . . . .                                                              | 2 |
| 3   | $n$ -step TD Control . . . . .                                                   | 4 |
| 3.1 | $n$ -step Sarsa . . . . .                                                        | 4 |
| 3.2 | $n$ -step Expected Sarsa . . . . .                                               | 5 |
| 3.3 | $n$ -step Off-policy Learning . . . . .                                          | 5 |
| 3.4 | Per-decision Methods with Control Variates . . . . .                             | 6 |
| 3.5 | Off-policy Learning without Importance Sampling: Backup Tree Algorithm . . . . . | 6 |
| 3.6 | $n$ -step $Q(\sigma)$ : A Unifying Algorithm . . . . .                           | 6 |

# 1 Introduction

The ***n*-step bootstrapping** method unifies the Monte Carlo method and the one-step temporal-difference method. Acutally, MC and TD are the opposite extreme of the *n*-step method.

## 2 *n*-step TD Prediction

### 2.1 *n*-step Return

The original TD prediction took only one step further for the update of value function. *n*-step TD prediction generalizes this, and enable to take steps more than one. That is, the target for the update is following ***n*-step return**;

#### Definition 2.1.1 (*n*-step Return)

Let  $R_t$  the reward on time step  $t$  and let  $V_t$  be the estimate of state-value function on time  $t$ . Then, the ***n*-step return**  $G_{t:t+n}$  is defined as

$$G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V_{t+n-1}(S_{t+n}). \quad (1.1)$$

for all  $n, t$  such that  $n \geq 1$  and  $0 \leq t < T - n$ . If  $t + n \geq T$ , then all the missing terms are taken as zero. That is,  $G_{t:t+n} = G_t$  if  $t + n \geq T$ .

All *n*-step returns can be considered as an approximation of full return. Note that using the definition above, 1-step return equals the target of the original one-step TD method, and  $\infty$ -step return equals the target of the MC method, the full return.

### 2.2 Algorithm

Using this *n*-step return, the *n*-step TD method use following update;

$$V_{t+n}(S_t) = V_{t+n-1}(S_t) + \alpha [G_{t:t+n} - V_{t+n-1}(S_t)] \quad (1.2)$$

while the values of all other states remain unchanged.

---

**Algorithm 1:**  $n$ -step TD Prediction

---

**Input :**  $\pi$  ..... policy  
 $\alpha \in (0, 1]$  ..... step size parameter  
 $n > 0$  ..... steps to take for update

**Output:**  $v_\pi$  ..... state-value function under policy  $\pi$

---

```
// initialize
1 foreach  $s \in \mathcal{S}$  do
2    $V(s) \leftarrow$  arbitrary
3 foreach episode do
4    $S_0 \leftarrow$  arbitrary state other than terminal state
5   store  $S_0$ 
6    $T \leftarrow \infty$ 
7   for  $t = 0, 1, 2, \dots$  do
8     if  $t < T$  then
9       take an action according to  $\pi(\cdot|S_t)$ 
10      observe and store the next reward as  $R_{t+1}$  and the next state as  $S_{t+1}$ 
11      if  $S_{t+1}$  is terminal then
12         $T \leftarrow t + 1$ 
13       $\tau \leftarrow t - n + 1$ 
14      if  $\tau = T - 1$  then
15        break
16      else if  $\tau \geq 0$  then
17         $G \leftarrow \begin{cases} \sum_{i=\tau+1}^{\tau+n} \gamma^{i-\tau-1} R_i + \gamma^n V(S_{\tau+n}) & \text{if } \tau + n < T \\ \sum_{i=\tau+1}^T \gamma^{i-\tau-1} R_i & \text{if } \tau + n \geq T \end{cases} \quad (G_{\tau:\tau+n})$ 
18         $V(S_\tau) \leftarrow V(S_\tau) + \alpha [G - V(S_\tau)]$ 
19 return  $V \simeq v_\pi$ 
```

---

The  $n$ -step TD prediction has an important property called **error reduction property**. The worst error of the expected  $n$ -step return is guaranteed to be less than or equal to  $\gamma^n$  times the worst error under  $V_{t+n-1}$ ;

**Definition 2.2.1 (Error Reduction Property)**

Let  $V_{t+n-1}$  be the  $n$ -step estimation of  $v_\pi$ , and let  $G_{t:t+n}$  be the  $n$ -step return. Then, following inequality holds;

$$\max_s \left| \mathbb{E}_\pi[G_{t:t+n} | S_t = s] - v_\pi(s) \right| \leq \gamma^n \max_s \left| V_{t+n-1}(s) - v_\pi(s) \right| \quad (1.3)$$

for all  $n \geq 1$ .

Since the estimate  $V_{t+n-1}$  targets  $G_{t:t+n}$  as one can see in the update equation 1.2, one can show formally that all  $n$ -step methods converge to the correct prediction, under appropriate technical conditions.

### 3 $n$ -step TD Control

#### 3.1 $n$ -step Sarsa

Instead of using state-value function, we use action-value function for the estimation. That is, we define the  $n$ -step return as

$$G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n Q_{t+n-1}(S_{t+n}, A_{t+n}) \quad (1.4)$$

for  $n \geq 1$  and  $0 \leq t < T - n$ . For  $t + n \geq T$ ,  $G_{t:t+n} = G_t$ . Then, the natural update for  $n$ -step Sarsa is

$$Q_{t+n}(S_t, A_t) = Q_{t+n-1}(S_t, A_t) + \alpha [G_{t:t+n} - Q_{t+n-1}(S_t, A_t)] \quad (1.5)$$

for  $0 \leq t < T$ , with values of other states remain unchanged.

---

**Algorithm 2:**  $n$ -step Sarsa

---

**Input :**  $\alpha \in (0, 1]$  ..... step size parameter  
 $0 < \varepsilon < 1$  ..... probability of exploring  
 $n$  ..... steps to take for update  
**Output:**  $q_*$  ..... the optimal action-value function

---

```

// initialize
1 foreach  $(s, a) \in \mathcal{S} \times \mathcal{A}$  do
2    $Q(s, a) \leftarrow$  arbitrary
3  $\pi \leftarrow \varepsilon$ -greedy policy with respect to  $Q$ 
4 foreach episode do
5    $S_0 \leftarrow$  arbitrary other than terminal state
6   store  $S_0$ 
7   select and store an action  $A_0 \sim \pi(\cdot | S_0)$ 
8    $T \leftarrow \infty$ 
9   for  $t = 0, 1, 2, \dots$  do
10    if  $t < T$  then
11      take action  $A_t$ 
12      observe and store the next reward as  $R_{t+1}$  and the next state as  $S_{t+1}$ 
13      if  $S_t$  is terminal then
14         $T \leftarrow t + 1$ 
15      else
16        select and store an action  $A_{t+1} \sim \pi(\cdot | S_{t+1})$ 
17     $\tau \leftarrow t - n + 1$ 
18    if  $\tau = T - 1$  then
19      break
20    else if  $\tau \geq 0$  then
21       $G \leftarrow \begin{cases} \sum_{i=\tau+1}^{\tau+n} \gamma^{i-\tau-1} R_i + \gamma^n Q(S_{\tau+n}, A_{\tau+n}) & \text{if } \tau + n < T \\ \sum_{i=\tau+1}^T \gamma^{i-\tau-1} R_i & \text{if } \tau + n \geq T \end{cases} \quad (G_{\tau:\tau+n})$ 
22       $Q(S_\tau, A_\tau) \leftarrow Q(S_\tau, A_\tau) + \alpha [G - Q(S_\tau, A_\tau)]$ 
23      If  $\pi$  is being learned, then ensure that  $\pi(\cdot | S_\tau)$  is  $\varepsilon$ -greedy with respect to  $Q$ 

```

---

### 3.2 $n$ -step Expected Sarsa

The  $n$ -step Expected Sarsa uses following  $n$ -step return;

$$G_{t:t+n} = R_{t+1} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n \bar{V}_{t+n-1}(S_{t+n}) \quad (1.6)$$

for  $t+n < T$ , with  $G_{t:t+n} = G_t$  for  $t+n > T$ . The  $\bar{V}_t(s)$  is the *expected approximate value* of state  $s$ , defined as

$$\bar{V}_t(s) = \sum_a \pi(a|s) Q_t(s, a) \quad (1.7)$$

for all  $s \in \mathcal{S}$  under the target policy  $\pi$ .

### 3.3 $n$ -step Off-policy Learning

Off-policy method uses two policies, the target policy  $\pi$  and behavior policy  $b$ . The behavior policy explores state-action pair and target policy learns the optimal policy of the problem. Since the expectation of the return depends on the behavior policy instead of the target policy, we multiply the *importance sampling ratio*, defined as  $\rho_{t:h} = \prod_{k=t}^{\min(h, T-1)} \frac{\pi(A_k|S_k)}{b(A_k|S_k)}$ . Then, the update is

$$V_{t+n}(S_t) = V_{t+n-1}(S_t) + \alpha \rho_{t:t+n-1} [G_{t:t+n} - V_{t+n-1}(S_t)] \quad (1.8)$$

for  $0 \leq t < T$ , and

$$Q_{t+n}(S_t, A_t) = Q_{t+n-1}(S_t, A_t) + \alpha \rho_{t+1:t+n} [G_{t:t+n} - Q_{t+n-1}(S_t, A_t)] \quad (1.9)$$

for  $0 \leq t < T$ .

---

**Algorithm 3:** Off-policy  $n$ -step Sarsa

---

**Input :**  $b$  ..... an arbitrary behavior policy such that  $b(a|s) > 0$  for all  $s \in \mathcal{S}, a \in \mathcal{A}$   
 $\alpha \in (0, 1]$  ..... step size parameter  
 $n$  ..... steps to take for update  
**Output:**  $q_*$  ..... optimal action-value function

---

```
// initialize
1 foreach  $(s, a) \in \mathcal{S} \times \mathcal{A}$  do
2    $Q(s, a) \leftarrow$  arbitrary
3 foreach episode do
4   initialize and store  $S_0$ , which is not terminal state
5   select and store an action  $A_0 \sim b(\cdot|S_0)$ 
6    $T \leftarrow \infty$ 
7   for  $t = 0, 1, 2, \dots$  do
8     if  $t < T$  then
9       take action  $A_t$ 
10      observe and store the next reward as  $R_{t+1}$  and the next state as  $S_{t+1}$ 
11      if  $S_{t+1}$  is terminal then
12         $T \leftarrow t + 1$ 
13      else
14        select and store an action  $A_{t+1} \sim b(\cdot|S_{t+1})$ 
15       $\tau \leftarrow t - n + 1$ 
16      if  $\tau = T - 1$  then
17        break
18      else if  $\tau \geq 0$  then
19         $\rho \leftarrow \prod_{i=\tau+1}^{\min(\tau+n-1, T-1)} \frac{\pi(A_i|S_i)}{b(A_i|S_i)}$ 
20         $G \leftarrow \begin{cases} \sum_{i=\tau+1}^{\tau+n} \gamma^{i-\tau-1} R_i + \gamma^n Q(S_{\tau+n}, A_{\tau+n}) & \text{if } \tau + n < T \\ \sum_{i=\tau+1}^T \gamma^{i-\tau-1} R_i & \text{if } \tau + n \geq T \end{cases} \quad (G_{\tau:\tau+n})$ 
21         $Q(S_\tau, A_\tau) \leftarrow Q(S_\tau, A_\tau) + \alpha \rho [G - Q(S_\tau, A_\tau)]$ 
22        If  $\pi$  is being learned, then ensure that  $\pi(\cdot|S_\tau)$  is greedy with respect to  $Q$ 
```

---

### 3.4 Per-decision Methods with Control Variates

### 3.5 Off-policy Learning without Importance Sampling: Backup Tree Algorithm

### 3.6 $n$ -step $Q(\sigma)$ : A Unifying Algorithm